Tutorials
C#
ASP.NET CORE MVC
PYTHON
JAVA
JAVASCRIPT
REACT JS
NODE JS
Salesforce
DEVOPS
Software Testing
Selenium JAVA
Selenium Python
Spring MVC & Boot
Angular 21.0

Classroom Training
Online Training

WEB API in ASP.NET CORE MVC | EXPLAIN WEB API IN DEPTH by Shiva Sir

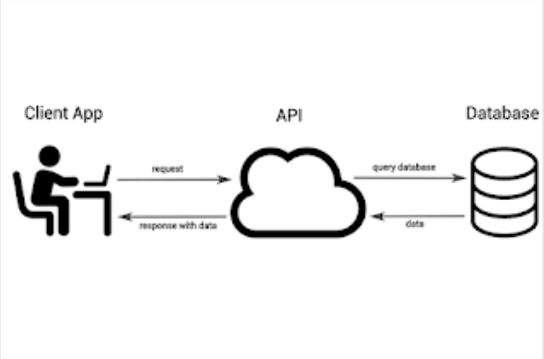
Shiva Gautam - August 01, 2025

1 What is Web API?

🔹 Definition:

A **Web API** (Web Application Programming Interface) is a **set of HTTP endpoints** exposed by a server application that allows other applications (frontend, mobile, etc.) to **communicate over the web**.

In .NET Core, a Web API is a type of controller that **returns data** (usually JSON), not views (HTML).



2 Why Use Web API?

Purpose	Description
🔄 Communication	Enables frontend and backend apps to communicate using HTTP
⚙️ Reusability	Backend APIs can be used by web apps, mobile apps, desktop apps
🔐 Centralized Business Logic	All logic is managed on the server side
🚀 Lightweight & Fast	Returns raw data (JSON), which is faster than rendering views

3 Web API vs MVC

Feature	MVC	Web API
Output	Returns Views (HTML)	Returns Data (JSON/XML)
Use Case	Web applications (UI required)	APIs for mobile, SPA, external systems

Feature	MVC	Web API
Controller Type	`Controller`	`ApiController`
Return Type	`IActionResult` with View	`ActionResult<T>` with data

Architecture of Web API in .NET Core

Layers in a Typical Web API Project:

1. **Model**
   - Represents the data structure
   - Example: `Student`, `Product`, `Employee`
2. **Controller**
   - Manages API endpoints
   - Handles HTTP requests (GET, POST, PUT, DELETE)
3. **DbContext (Data Access Layer)**
   - Entity Framework Core class for database communication
4. **Database**
   - SQL Server, PostgreSQL, etc.

Request Lifecycle:

1. Client sends an HTTP request → `https://api.site.com/products`
2. Router matches the endpoint → `ProductController`
3. Action method is called → `GetAllProducts()`
4.

Data is fetched from the database via `DbContext`

5.

Data is returned as JSON using `Ok(result)`

6.

Client (e.g., React app or Postman) receives the JSON response

5 Key Attributes in Web API

Attribute	Description
`[ApiController]`	Specifies the class is a Web API controller. Adds automatic model validation and behavior
`[Route()]`	Defines the URL path to reach the action method
`[HttpGet]`	Handles GET requests
`[HttpPost]`	Handles POST (create) requests
`[HttpPut]`	Handles PUT (update) requests
`[HttpDelete]`	Handles DELETE requests

6 Return Types in Web API

Return Type	Description
`IActionResult`	Base type for any HTTP response
`ActionResult<T>`	Strongly typed return data (like `ActionResult<Student>`)
`Ok(data)`	Returns 200 OK with data
`NotFound()`	Returns 404
`BadRequest()`	Returns 400
`CreatedAtAction()`	Returns 201 with location header

7 RESTful API Principles

Web APIs in .NET Core follow **REST** principles (REpresentational State Transfer):

Method	Action	Purpose
GET	Read	Get data from server
POST	Create	Send new data to server
PUT	Update	Modify existing data
DELETE	Delete	Remove data

8 Entity Framework Core in Web API

- EF Core is the ORM used to map C# objects to database tables.
-

Benefits:

- Write LINQ instead of SQL
- Manage migrations
- Easy CRUD with `DbSet<T>`

9️⃣ Dependency Injection

- Web API uses **Dependency Injection (DI)** to manage services like `DbContext`.
- Automatically injects services into controllers.

```
csharp

public StudentController(AppDbContext context)
{
    _context = context;
}
```

🔒 1️⃣0️⃣ Security in Web API (Basics)

Mechanism	Purpose
API Key	Authenticate client requests
JWT (Token)	Secure APIs using bearer tokens
HTTPS	Encrypts data between client and server
CORS	Controls which frontend apps can access your API

1️⃣1️⃣ Middleware in Web API

- Middlewares are used to process requests/responses.
- Example: Authentication, Logging, Exception handling

In `Program.cs`:

```
csharp

app.UseAuthentication();
app.UseAuthorization();
app.UseCors();
```

1️⃣2️⃣ Versioning in Web API (Optional but Useful)

Helps manage changes over time:

```
csharp

[Route("api/v1/[controller]")]
```

[ApiVersion("1.0")]

EXAMPLE OF API:

🔧 Step-by-Step Implementation:

✅ Step 2: Create Project

1. Open Visual Studio
2. Create a new project → **ASP.NET Core Web API**
3. Name: `StudentAPI`
4. Choose .NET 6/7 → Click Create

✅ Step 3: Create Student Model

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
    public string Course { get; set; }
}
```

✅ Step 4: Create DbContext

Install EF Core packages:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Now, create `AppDbContext.cs`:

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Student> Students { get; set; }
}
```

✅ Step 5: Configure Database Connection

In `appsettings.json`:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=.;Database=StudentDB;Trusted_Connection=True;"
}
```

In `Program.cs`:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllers();
```

✅ Step 6: Create Student Controller

`Controllers/StudentController.cs`

```
[ApiController]
[Route("api/[controller]")]
public class StudentController : ControllerBase
{
    private readonly AppDbContext _context;

    public StudentController(AppDbContext context)
    {
        _context = context;
    }

    // GET: api/student
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Student>>> GetAll()
    {
        return await _context.Students.ToListAsync();
    }

    // GET: api/student/1
    [HttpGet("{id}")]
    public async Task<ActionResult<Student>> Get(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student == null) return NotFound();
        return student;
    }

    // POST: api/student
    [HttpPost]
    public async Task<ActionResult<Student>> Create(Student student)
    {
        _context.Students.Add(student);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(Get), new { id = student.Id }, student);
    }

    // PUT: api/student/1
    [HttpPut("{id}")]
    public async Task<IActionResult> Update(int id, Student student)
    {
        if (id != student.Id) return BadRequest();

        _context.Entry(student).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return NoContent();
    }

    // DELETE: api/student/1
    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student == null) return NotFound();

        _context.Students.Remove(student);
        await _context.SaveChangesAsync();
        return NoContent();
```

```
    }
}
```

✅ Step 7: Run Migrations and Update DB

```
Add-Migration InitialCreate
Update-Database
```

This creates the `StudentDB` database and `Students` table.

✅ Step 8: Test API Using Postman

Base URL: `https://localhost:5001/api/student`

Operation	HTTP Verb	URL	Body (if needed)
Get All Students	GET	`/api/student`	–
Get by ID	GET	`/api/student/1`	–
Create	POST	`/api/student`	JSON: `{"name":"Aman","age":21,"course":"C#"}`
Update	PUT	`/api/student/1`	JSON: updated values
Delete	DELETE	`/api/student/1`	–

🏁 Final Structure:

```
StudentAPI/
│
├── Controllers/
│   └── StudentController.cs
│
├── Models/
│   └── Student.cs
│
├── Data/
│   └── AppDbContext.cs
│
├── appsettings.json
└── Program.cs
```

🧠 Is Web API REST or SOAP?

✅ Short Answer:

A **Web API** can be **either REST or SOAP** – **it depends on how the API is designed**.

🔍 Let's Understand:

🔹 Web API

-

A **general term** for any API that can be accessed using the web (HTTP/HTTPS).

-

It is a **technology-neutral** term.

- It could be implemented using **REST**, **SOAP**, **GraphQL**, **gRPC**, etc.

✅ Common Types of Web APIs:

Type	Description
REST API	Most common. Uses HTTP verbs (GET, POST, etc.), returns JSON or XML, stateless.
SOAP API	Older, strict protocol. Uses XML, defined by WSDL, uses POST only.
GraphQL API	Modern alternative to REST, client controls the data shape.
gRPC API	High-performance, binary-based, good for microservices.

📌 Example in .NET Core:

- If you create a **Web API using ASP.NET Core**, it's by default **a RESTful Web API**.
  - Uses `[ApiController]`, routes like `/api/products`, and returns JSON.
  - Uses HTTP methods like GET, POST, PUT, DELETE.
- If you want to create a **SOAP-based API**, you would use **WCF (Windows Communication Foundation)** — not Web API.

Entity Framework Core Code First Example (Department & Employee)

This example demonstrates a Code First approach in .NET 8/9 Entity Framework Core with the following constraints:
✅ Primary Key
✅ Foreign Key
✅ NOT NULL
✅ Maximum Length
✅ Required Fields
✅ Default Value
✅ Unique Constraint
✅ Check Constraint
✅ One-to-Many Relationship
Project Structure

```
Models
│
├── Department.cs
├── Employee.cs
│
Data
│
└── AppDbContext.cs
```

Department Entity

```
using System.ComponentModel.DataAnnotations;
public class Department
```

```
{
  [Key]
  public int DepartmentId { get; set; }
  [Required]
  [StringLength(100)]
  public string DepartmentName { get; set; } = string.Empty;
  [StringLength(250)]
  public string? Description { get; set; }
  public ICollection<Employee> Employees { get; set; }
    = new List<Employee>();
}
```

Employee Entity

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
public class Employee
{
  [Key]
  public int EmployeeId { get; set; }
  [Required]
  [StringLength(100)]
  public string EmployeeName { get; set; } = string.Empty;
  [Required]
  [EmailAddress]
  [StringLength(150)]
  public string Email { get; set; } = string.Empty;
  [Required]
  [StringLength(15)]
  public string MobileNo { get; set; } = string.Empty;
  [Required]
  [Column(TypeName = "decimal(18,2)")]
  public decimal Salary { get; set; }
  [Required]
  public DateTime JoiningDate { get; set; }
  public bool IsActive { get; set; }
  //————————————-
  // Foreign Key
  //————————————-
  [ForeignKey("Department")]
  public int DepartmentId { get; set; }
  public Department Department { get; set; } = null!;
}
```

AppDbContext

```
using Microsoft.EntityFrameworkCore;
public class AppDbContext : DbContext
{
  public AppDbContext(DbContextOptions<AppDbContext> options)
    : base(options)
  {
  }
  public DbSet<Employee> Employees => Set<Employee>();
  public DbSet<Department> Departments => Set<Department>();
  protected override void OnModelCreating(ModelBuilder modelBuilder)
  {
    base.OnModelCreating(modelBuilder);
    //————————————————-
    // Department
    //————————————————-
    modelBuilder.Entity<Department>()
      .HasIndex(x => x.DepartmentName)
      .IsUnique();
    //————————————————-
```

```
    // Employee
    //————————————————————
    modelBuilder.Entity<Employee>()
      .HasIndex(x => x.Email)
      .IsUnique();
    modelBuilder.Entity<Employee>()
      .Property(x => x.IsActive)
      .HasDefaultValue(true);
    modelBuilder.Entity<Employee>()
      .HasCheckConstraint("CK_Employee_Salary",
                "Salary >= 10000");
    //————————————————————
    // One-To-Many Relationship
    //————————————————————
    modelBuilder.Entity<Employee>()
      .HasOne(e => e.Department)
      .WithMany(d => d.Employees)
      .HasForeignKey(e => e.DepartmentId)
      .OnDelete(DeleteBehavior.Restrict);
  }
}
```

Connection String

appsettings.json

```
{
 "ConnectionStrings": {
  "DefaultConnection": "Server=.;Database=CompanyDB;Trusted_Connection=True;TrustServerCertificate=True;"
 }
}
```

Register DbContext

```
builder.Services.AddDbContext<AppDbContext>(options =>
{
  options.UseSqlServer(
    builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

Create Migration

Add-Migration InitialCreate

or

dotnet ef migrations add InitialCreate

Update Database

Update-Database

SQL Generated by EF Core (Equivalent)

Department Table

```
CREATE TABLE Departments
(
  DepartmentId INT IDENTITY(1,1) PRIMARY KEY,
  DepartmentName NVARCHAR(100) NOT NULL UNIQUE,
  Description NVARCHAR(250) NULL
)
```

Employee Table

```
CREATE TABLE Employees
(
  EmployeeId INT IDENTITY(1,1) PRIMARY KEY,
  EmployeeName NVARCHAR(100) NOT NULL,
  Email NVARCHAR(150) NOT NULL UNIQUE,
  MobileNo NVARCHAR(15) NOT NULL,
  Salary DECIMAL(18,2) NOT NULL
    CHECK(Salary>=10000),
  JoiningDate DATETIME2 NOT NULL,
  IsActive BIT NOT NULL DEFAULT(1),
  DepartmentId INT NOT NULL,
  CONSTRAINT FK_Department
```

```
    FOREIGN KEY(DepartmentId)
    REFERENCES Departments(DepartmentId)
)
```

Insert Sample Data

```
var department = new Department
{
    DepartmentName = "IT",
    Description = "Software Development"
};
_context.Departments.Add(department);
await _context.SaveChangesAsync();
var employee = new Employee
{
    EmployeeName = "Shiva",
    Email = "shiva@gmail.com",
    MobileNo = "9876543210",
    Salary = 60000,
    JoiningDate = DateTime.Now,
    DepartmentId = department.DepartmentId
};
_context.Employees.Add(employee);
await _context.SaveChangesAsync();
Retrieve Data with LINQ
var employees = await _context.Employees
            .Include(e => e.Department)
            .ToListAsync();
foreach (var emp in employees)
{
    Console.WriteLine($"{emp.EmployeeName} - {emp.Department.DepartmentName}");
}
```

Relationship Diagram

```
       Department
+-----------------+
| DepartmentId (PK)       |
| DepartmentName (UNIQUE)    |
| Description             |
+-----------------+
         |
         | One
         |
         |
         Many
+-----------------+
| EmployeeId (PK)         |
| EmployeeName            |
| Email (UNIQUE)          |
| MobileNo                |
| Salary                  |
| JoiningDate             |
| IsActive                |
| DepartmentId (FK)       |
+-----------------+
Constraints Used
Constraint Example
Primary Key     EmployeeId, DepartmentId
Foreign Key     DepartmentId → Departments.DepartmentId
```

NOT NULL [Required]
Maximum Length [StringLength(100)]
Unique HasIndex(...).IsUnique()
Default Value HasDefaultValue(true)
Check Constraint Salary >= 10000
One-to-Many One Department → Many Employees

Tags ASP.NET CORE MVC

Facebook Twitter

OLDER
What is Jenkins? How to create pipeline using jenkins

NEWER
What is Maven | How to create MAVEN Environment in details



POSTED BY SHIVA GAUTAM
Hi, I'm Shiva Gautam. With over 15 years of diverse experience in various IT domains, I am now an entrepreneur focusing on both training and software development. My expertise spans across multiple technologies, and I've authored numerous articles on subjects including Java, Python, REACT, NODE, ANGULAR, Cloud, software testing, Django framework, C#, Flutter, Salesforce, Ruby on Rails, .NET, AWS, DEVOPS, AI/ML and more. For additional information, please visit shivaconceptsolution.com and kangaroosoftware.net.

You may like these posts

ASP.NET CORE MVC
Caching in ASP.NET Core Web API — Complete Tutorial
August 20, 2026

ASP.NET CORE MVC
Entity Framework Core with StoredProcedure Example in Code FIrst And Data First Approach
August 06, 2026

ASP.NET CORE MVC
Best AI Integration Tools in Visual Studio 2022 (2026 Guide)
May 19, 2026

Post a Comment

0 Comments

POST Answer of Questions and ASK to Doubt

SHIVA TUTORIAL

- Tutorials
- C#
- ASP.NET CORE MVC
- PYTHON
- JAVA
- JAVASCRIPT
- REACT JS
- NODE JS
- Salesforce
- DEVOPS
- Software Testing
- Selenium JAVA
- Selenium Python
- Spring MVC & Boot
- Angular 21.0

- Classroom Training
- Online Training

✨ JOIN AWS, DEVOPS, SALESFORCE, DATA ANALYTICS, JAVA, .NE

Get 75% Discount Early bird offer CLICK to JOIN 100% Placement Oriented P

Created By Blogger Theme | Distributed By Gooyaabi Templates